

JAVA LAB RECORD

Experiment No: 06 Date: 23/09/26

Title: Implementation of TreeSet Methods in Java

1. Objective / Aim

To design and implement a Java program using the TreeSet collection and demonstrate its commonly used methods such as add(), contains(), first(), last(), higher(), lower(), ceiling(), floor(), remove(), pollFirst(), and pollLast().

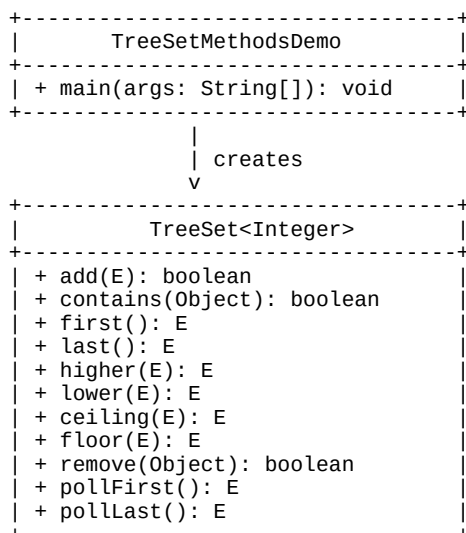
2. System Requirements

Hardware Requirements	Software Requirements
• Computer/Laptop • Minimum 4 GB RAM • Keyboard and Mouse • At least 500 MB free storage	• Operating System: Windows / Linux / macOS • JDK 17 or JDK 21 • IDE: Eclipse / IntelliJ IDEA / NetBeans / VS Code • Command Prompt / Terminal

3. Algorithm / Logic Description

1. Import the TreeSet class from java.util package.
2. Create a TreeSet named numbers.
3. Add the values 40, 10, 30, 20, and 50. TreeSet automatically stores them in ascending order.
4. Use contains() to check whether 30 and 60 are present.
5. Use first() and last() to display the smallest and largest elements.
6. Use higher(30) and lower(30) to find the nearest greater and smaller elements.
7. Use ceiling(25) and floor(25) to find the smallest element greater than or equal to 25 and the largest element less than or equal to 25.
8. Remove 20 using remove().
9. Use pollFirst() to remove and return the smallest element.
10. Use pollLast() to remove and return the largest element.
11. Display the TreeSet after each required operation.

4. UML Class Diagram



5. Source Code

```
import java.util.TreeSet;

class TreeSetMethodsDemo {

    public static void main(String[] args) {

        // Creating a TreeSet
        TreeSet<Integer> numbers = new TreeSet<>();

        // add(E e)
        numbers.add(40);
        numbers.add(10);
        numbers.add(30);
        numbers.add(20);
        numbers.add(50);

        System.out.println("After add(): " + numbers);

        // contains(Object o)
        System.out.println("Contains 30? " + numbers.contains(30));
        System.out.println("Contains 60? " + numbers.contains(60));

        // first()
        System.out.println("First element: " + numbers.first());

        // last()
        System.out.println("Last element: " + numbers.last());

        // higher(E e)
        System.out.println("Element higher than 30: " + numbers.higher(30));

        // lower(E e)
        System.out.println("Element lower than 30: " + numbers.lower(30));

        // ceiling(E e)
        System.out.println("Ceiling of 25: " + numbers.ceiling(25));

        // floor(E e)
        System.out.println("Floor of 25: " + numbers.floor(25));

        // remove(Object o)
        numbers.remove(20);
        System.out.println("After remove(20): " + numbers);

        // pollFirst()
        System.out.println("Removed first element: " + numbers.pollFirst());
        System.out.println("After pollFirst(): " + numbers);

        // pollLast()
        System.out.println("Removed last element: " + numbers.pollLast());
        System.out.println("After pollLast(): " + numbers);
    }
}
```

6. Completion and Execution Commands

1. Save the program as: TreeSetMethodsDemo.java
2. Compile the program:
javac TreeSetMethodsDemo.java
3. Run the program:
java TreeSetMethodsDemo

7. Output

```
After add(): [10, 20, 30, 40, 50]
Contains 30? true
Contains 60? false
First element: 10
Last element: 50
Element higher than 30: 40
Element lower than 30: 20
Ceiling of 25: 30
Floor of 25: 20
After remove(20): [10, 30, 40, 50]
Removed first element: 10
After pollFirst(): [30, 40, 50]
Removed last element: 50
```

After pollLast(): [30, 40]

8. Result

The TreeSet program was successfully implemented and executed. The program demonstrates sorted, unique elements and the major navigation, search, removal, and polling operations provided by TreeSet.